PROJECT PROPOSAL

File Deduplication and Storage Optimization Using Advanced Data Structures

Submitted By:
Aryan Ravi Zodape 206126006
Jyotiraditya Kumar 206126014

Introduction

The proposed project is a **File Deduplication and Storage Optimization System** using C++. The system will identify duplicate files in a storage system and reduce unnecessary storage by maintaining a single copy of identical file contents. The main focus of the project will be on the implementation and application of advanced data structures for efficient duplicate detection and storage management.

The system will generate fingerprints for files and use them to efficiently identify potential duplicates. A multi-stage verification process using file size, hashing, and content comparison will be used to ensure reliable duplicate detection. The project will also investigate probabilistic and hierarchical data structures to improve performance for large collections of files.

Objectives

- To design and implement an efficient file deduplication system using C++.
- To identify duplicate files using hashing and efficient indexing techniques.
- To implement a Hash Table for fast fingerprint-based file lookup.
- To implement a Bloom Filter for reducing unnecessary hash table lookups.
- To use a Merkle Tree for hierarchical representation and efficient change detection.
- To analyze time complexity, space complexity, and storage savings achieved through deduplication.
- To compare performance under varying dataset sizes and duplicate ratios.

Data Structures and Algorithms Used

- **Hash Table** – for mapping file fingerprints to files and enabling fast duplicate lookup.
- **Bloom Filter** – for quickly determining whether a file fingerprint may already exist.
- **Merkle Tree** – for hierarchical representation of file or directory contents and efficient change detection.
- **Hashing** – for generating fingerprints of file contents.
- **Heap / Priority Queue** – optionally for identifying duplicate groups with the highest storage savings.

Proposed System

File Scanner → **Hash Generator** → **Bloom Filter** → **Hash Table** → **Duplicate Verification** → **Storage Optimizer**

For each file, metadata and a content fingerprint will be generated. The Bloom Filter will first determine whether the fingerprint is potentially known. If so, the Hash Table will be queried to locate candidate duplicates. Candidate files will then be verified using file size and content comparison. Confirmed duplicates can be represented as references to a single stored copy, thereby reducing storage consumption.

Expected Outcome

The project will result in a functional C++ file deduplication and storage optimization system. Experimental evaluation will measure duplicate detection time, indexing time, memory consumption, number of file comparisons, Bloom Filter false-positive rate, and storage savings. The project will demonstrate how appropriate data structures can improve the efficiency of large-scale duplicate detection and storage management.